

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Adrian Mutu

Group: FAF-242

Report

Laboratory Work No. 2

Operating Systems

Topic: Meet the OS You Will Build

Checked by:

Patricia Reitman

FCIM, UTM

Chișinău – 2026

Contents

1 Purpose of the Laboratory Work **3**

2 Part 1 & 2: Running and Exploring xv6 **3**

2.1 Booting the kernel and commands executed inside xv6 3

2.2 Observations (Part 2) 3

3 Part 3: Source Code Analysis **4**

3.1 Inspecting the system calls 4

3.2 Observations (Part 3) 4

4 Part 4: Implementing the sleep Utility **5**

4.1 Source code: user/sleep.c 5

4.2 Configuring the build in the Makefile 5

4.3 Testing the program 5

1 Purpose of the Laboratory Work

To become familiar with the educational operating system **xv6** (a re-implementation of Unix Version 6 for the RISC-V architecture) running inside the QEMU emulator. The laboratory involves compiling the kernel, exploring the runtime environment, analysing the source code in the `kernel/` and `user/` directories, and implementing the first user-space utility program: `sleep.c`.

The work was carried out on an Ubuntu 26.04 (aarch64) virtual machine. The toolchain used was `gcc-riscv64-linux-gnu` together with `qemu-system-riscv64`; the source tree was obtained with `git clone https://github.com/mit-pdos/xv6-riscv.git` and the system was built and started with `make qemu`.

2 Part 1 & 2: Running and Exploring xv6

2.1 Booting the kernel and commands executed inside xv6

```
$ make qemu
...
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
...
wc             2 17 37728
console        3 23 0

$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...

$ echo hello xv6
hello xv6

$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0

$ wc README
48 336 2441 README
```

2.2 Observations (Part 2)

1. **Three programs shipped with xv6:** `ls`, `cat`, `echo` (as well as `grep`, `wc` and the shell `sh`).
2. **Two essential OS features required for pipes to work:**

• Support for multiple processes and concurrent execution (processes are instantiated with `fork` and scheduled by the kernel).

• The inter-process communication (IPC) mechanism managed in the kernel through file descriptors and the redirection of data streams.

3. Comparison between the xv6 shell and the Linux shell: The xv6 shell is a minimal, didactic implementation that provides the fundamental facilities (command execution, I/O redirection, and pipes), but lacks the modern features of Bash such as command history, Tab auto-completion, or advanced environment variables.

3 Part 3: Source Code Analysis

3.1 Inspecting the system calls

To analyse the way user-space programs invoke the kernel, the following commands were executed in the host directory `~/xv6-riscv`:

```
$ sed -n "1,30p" user/cat.c
$ cat user/user.h
$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

An excerpt from `user/cat.c` showing the system calls it relies on:

```
char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

3.2 Observations (Part 3)

1. System calls used in `user/cat.c`:

• `read()`: asks the kernel to read a stream of bytes from a file descriptor into a buffer in the process's memory.

• `write()`: asks the kernel to transfer the bytes from a buffer to a file descriptor (e.g. descriptor 1 for *stdout*).

• `exit()`: notifies the kernel that the current process has finished executing, releasing its associated structures and resources (the program also uses `open()` and `close()` when file arguments are given).

2. The implementation of `sys_read`: The function is implemented in the file `kernel/sysfile.c`, starting at line **69** (`sys_write` follows at line **83**).

3. The difference between the code in `kernel/` and `user/`: The code in `kernel/` runs in privileged *supervisor mode*, with direct access to hardware registers and physical memory, whereas the code in `user/` runs in unprivileged *user mode*, its interaction with resources being strictly mediated through system calls.

4 Part 4: Implementing the sleep Utility

4.1 Source code: user/sleep.c

The `sleep` utility program was created; it receives the number of clock ticks as an argument and invokes the existing system call. Note: in the current version of the xv6 source tree, the kernel system call that suspends a process was renamed from `sleep` to `pause` (`SYS_pause`, number 13, declared as `int pause(int);` in `user/user.h`), precisely so that the user utility can bear the name `sleep`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

4.2 Configuring the build in the Makefile

In the `Makefile`, the binary was added to the `UPROGS` list:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    $U/_forktest\
    $U/_grep\
    ...
```

4.3 Testing the program

After rebuilding the file-system image (`make fs.img` / `make qemu`), the behaviour was tested in the xv6 console with the following commands:

```
$ sleep 10
$ sleep
usage: sleep <ticks>
$ ls | grep sleep
sleep          2 5 35280
```

The command `sleep 10` suspended execution for the specified duration (10 clock ticks — about one second, since a tick is roughly 100 ms), returning to the prompt afterwards; invoking it with no parameters correctly triggered the explanatory usage error, and the binary is visible in the file system listing, confirming the `UPROGS` integration.